

Instructivo TSRC: Cómo modificar archivos y subirlos a GitHub sin sufrir

La idea básica

Tu repo tiene 2 “lugares”:

- **Tu PC** (local): donde editas archivos.
- **GitHub** (remoto): donde se publica y Cloudflare Pages despliega.

Git es el puente. Y tú siempre vas a hacer este ciclo:

Actualizar → Cambiar → Revisar → Guardar (commit) → Subir (push)

0) Reglas de oro

1. **Antes de empezar a editar**, siempre:

```
cd C:\Users\llano\Repos\stellarregistrycommittee  
git pull --rebase
```

2. **Antes de subir**, siempre revisa:

```
git status
```

3. **Nunca hagas pull --rebase con cambios sueltos** (Git te va a regañar).
Si tienes cambios, o los **commiteas**, o los **stashes** (los guardas en cajón).
-

Caso A: Modifiqué “código o templates” (schema/manifest/layout/render...)

Paso 1) Ver qué cambió

```
git status
```

Opcional (muy útil):

```
git diff
```

Paso 2) Seleccionar qué vas a guardar (add)

Tú eliges los archivos exactos:

```
git add tools/registrar/registrar/schema.py  
git add tools/tsrc/certificates/templates/tsrc-letter-v1/manifest.json
```

Si quieres agregar **todo** lo modificado (no siempre recomendado):

```
git add -A
```

Paso 3) Guardar checkpoint (commit)

```
git commit -m "Describe lo que cambiaste"
```

Ejemplos buenos:

- "Registrar: add certificate legend and template id"
- "Certificates: add fixed strings to manifest"
- "Vizier: enrich SAO metadata fields"

Paso 4) Subirlo a GitHub (push)

git push origin main

Paso 5) Confirmar que ya quedó limpio

git status

Debe decir:

nothing to commit, working tree clean

Caso B: Solo generé una entrada nueva (JSON) con el Registrador

Tu registrador hace:

- crea el JSON
- y si todo va bien: git add/commit/push de **ese archivo**

Aun así, tú revisa:

git status

Si quedó "clean", listo.

Si quedaron otros modified: (como schema.py, manifest.json) → esos **no se subieron** por el registrador, y los debes commitear tú (Caso A).

Problemas típicos y cómo se arreglan

Problema 1) "fetch first / remote is ahead"

Mensaje típico:

rejected (fetch first)

remote contains work that you do not have locally

Significa:

GitHub tiene cambios nuevos y tu PC está atrasada.

Solución:

1. Si git status está limpio:

```
git pull --rebase
git push
```

2. Si **NO está limpio** (tienes “modified”):
 - Opción A: haz commit de tus cambios, luego pull/push.
 - Opción B: usa stash (cajón), pull, y luego pop.

Problema 2) “cannot pull with rebase: You have unstaged changes”

Significa:

Tienes cambios sueltos; Git no quiere reacomodar historial así.

Solución rápida (stash):

```
git stash push -u -m "WIP before pull"
git pull --rebase
git stash pop
```

Problema 3) “Hice cambios pero no quería”

Si quieres **tirar los cambios** y volver al último commit:

```
git restore <archivo>
```

Ejemplo:

```
git restore tools/registrar/registrar/schema.py
```

Si el archivo era nuevo y no lo quieres:

```
del ruta\al\archivo
```

Cómo separar commits (muy recomendado)

Si tocaste cosas distintas, haz commits separados:

Commit 1: registrar

```
git add tools/registrar/registrar/schema.py tools/registrar/registrar/vizier.py
tools/registrar/app.py
git commit -m "Registrar: enrich SAO metadata and certificate legend"
git push
```

Commit 2: template

```
git add tools/tsrc/certificates/templates/tsrc-letter-v1/manifest.json
tools/tsrc/certificates/templates/tsrc-letter-v1/layout.json
```

```
git commit -m "Certificates: update template strings and layout"  
git push
```

Esto te salva de “mezclas raras” y hace el historial más claro.

Mini-cheatsheet (lo mínimo que debes memorizar)

- Ver estado: git status
- Ver cambios: git diff
- Seleccionar archivos: git add archivo
- Guardar: git commit -m "mensaje"
- Subir: git push
- Traer lo nuevo: git pull --rebase
- Guardar en cajón: git stash push -u -m "WIP"
- Sacar del cajón: git stash pop

Rutina recomendada (la que siempre funciona)

Antes de trabajar

```
cd C:\Users\llano\Repos\stellarregistrycommittee  
git pull --rebase  
git status
```

Después de trabajar

```
git status  
git add <archivos>  
git commit -m "mensaje"  
git push origin main  
git status
```